

Taller: Comunicación Segura — TLS sobre TCP vs DTLS sobre UDP

Curso: Sistemas Distribuidos

Duración: 120 minutos

Modalidad: Parejas o individual

Prerrequisito: Taller de Sockets TCP y UDP con Wireshark

Objetivos

Al finalizar este taller, el estudiante será capaz de:

- Generar certificados autofirmados con OpenSSL para uso en laboratorio.
- Implementar un servidor y un cliente TLS (TCP seguro) en Python.
- Implementar un servidor y un cliente DTLS (UDP seguro) en Python.
- Capturar el tráfico de ambos protocolos con Wireshark e identificar los mensajes del handshake.
- Comparar el tráfico seguro con el tráfico en texto plano del taller anterior.
- Explicar por qué los datos dejan de ser legibles en la traza y qué información sigue siendo visible.

1. Contexto

En el taller anterior, implementamos servidores y clientes TCP y UDP que enviaban mensajes en texto plano. Al capturar el tráfico con Wireshark, los mensajes eran completamente legibles — cualquier persona con acceso a la red podía leerlos.

En este taller vamos a agregar una capa de seguridad:

- **TLS (Transport Layer Security)** protege conexiones TCP. Es el protocolo detrás de HTTPS.
- **DTLS (Datagram Transport Layer Security)** protege comunicación UDP. Es esencialmente TLS adaptado para datagramas, donde los paquetes pueden perderse o llegar fuera de orden.

Vamos a reutilizar la estructura básica del taller anterior (mismo formato de mensajes, mismo puerto base) para que la comparación en Wireshark sea directa.

2. Preparación del entorno

Herramientas necesarias (además de las del taller anterior):

- OpenSSL instalado (`openssl version` para verificar). Se requiere versión 1.1.1 o superior.
- Python 3.8 o superior con el módulo `ssl` (viene incluido en la biblioteca estándar).

Generar los certificados:

Antes de comenzar con el código, necesitamos un certificado y una clave privada. En un entorno real estos serían emitidos por una Autoridad Certificadora (CA). Para el laboratorio usaremos un certificado autofirmado.

Ejecute el siguiente comando en la terminal:

```
openssl req -x509 -newkey rsa:2048 -keyout clave.pem -out certificado.pem -
days 365 -nodes \
  -subj "/C=CO/ST=Santander/L=Bucaramanga/O=Universidad/CN=localhost"
```

Esto genera dos archivos:

- **clave.pem** — la clave privada del servidor (nunca se comparte).
- **certificado.pem** — el certificado público (se comparte con los clientes).

Nota: La opción **-nodes** significa "no DES", es decir, la clave privada no se protege con contraseña. Esto es aceptable solo en un entorno de laboratorio.

Verifique que los archivos se crearon correctamente:

```
openssl x509 -in certificado.pem -text -noout | head -20
```

Debería ver información como el emisor, la validez y el algoritmo de firma.

3. Parte 1 — TLS sobre TCP

3.1 Qué esperar ver en Wireshark

Comparado con el taller anterior, la secuencia de TCP sigue siendo visible (SYN, SYN-ACK, ACK), pero después del handshake TCP aparece un **handshake TLS** adicional antes de que se transmitan datos. En Wireshark verá algo como:

CLIENTE	SERVIDOR
- SYN -> handshake TCP	
< SYN-ACK (igual que antes)	
- ACK ->	
- ClientHello --> handshake TLS	
< ServerHello, Certificate -- (nuevo)	
< ServerKeyExchange, Done	
- ClientKeyExchange -->	
- ChangeCipherSpec, Finished -->	
< ChangeCipherSpec, Finished	
- Application Data --> datos encriptados	
< Application Data (ilegibles)	

Los mensajes de datos ahora aparecen como "Application Data" y su contenido **no es legible** en Wireshark.

3.2 Servidor TLS

Cree el archivo `tls_servidor.py`:

```
import socket
import ssl

HOST = '127.0.0.1'
PORT = 9001 # puerto diferente al taller anterior para poder comparar

def crear_contexto_tls():
    contexto = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
    contexto.load_cert_chain('certificado.pem', 'clave.pem')
    return contexto

def manejar_cliente(conn_segura, addr):
    print(f'Conexión TLS desde {addr}')
    print(f' Versión: {conn_segura.version()}')
    print(f' Cipher: {conn_segura.cipher()}')
    with conn_segura:
        while True:
            datos = conn_segura.recv(1024)
            if not datos:
                break
            mensaje = datos.decode('utf-8')
            print(f'Recibido: {mensaje}')
            respuesta = f'Servidor recibio: {mensaje}'
            conn_segura.sendall(respuesta.encode('utf-8'))
    print('Conexión cerrada')

def main():
    contexto = crear_contexto_tls()
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen(1)
        print(f'Servidor TLS escuchando en {HOST}:{PORT}')
        with contexto.wrap_socket(s, server_side=True) as s_seguro:
            while True:
                conn_segura, addr = s_seguro.accept()
                manejar_cliente(conn_segura, addr)

if __name__ == '__main__':
    main()
```

3.3 Cliente TLS

Cree el archivo `tls_cliente.py`:

```
import socket
import ssl
import time
```

```

SERVIDOR = '127.0.0.1'
PORT = 9001

def main():
    contexto = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    contexto.load_verify_locations('certificado.pem')
    contexto.check_hostname = True

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        with contexto.wrap_socket(s, server_hostname='localhost') as
s_seguro:
            s_seguro.connect((SERVIDOR, PORT))
            print(f'Conectado con TLS')
            print(f' Versión: {s_seguro.version()}')
            print(f' Cipher: {s_seguro.cipher()}')

            mensajes = [
                'Hola, este es el primer mensaje seguro',
                'Este mensaje viaja encriptado',
                'Ni Wireshark puede leer esto',
                'Ultimo mensaje, adios'
            ]

            for msg in mensajes:
                s_seguro.sendall(msg.encode('utf-8'))
                respuesta = s_seguro.recv(1024).decode('utf-8')
                print(f'Enviado: {msg}')
                print(f'Respuesta: {respuesta}')
                time.sleep(1)

            print('Conexión cerrada')

if __name__ == '__main__':
    main()

```

3.4 Instrucciones de ejecución y captura

1. Abra Wireshark y seleccione la interfaz **lo** (loopback) o la interfaz de red correspondiente.
2. Aplique el filtro de captura: **tcp port 9001**.
3. Inicie la captura.
4. En una terminal, ejecute el servidor: **python3 tls_servidor.py**.
5. En otra terminal, ejecute el cliente: **python3 tls_cliente.py**.
6. Cuando el cliente termine, detenga la captura.
7. Guarde la captura como **captura_tls.pcap**.

Filtros útiles para Wireshark:

Filtro	Qué muestra
tls	Solo paquetes TLS

Filtro	Qué muestra
<code>tls.handshake</code>	Solo mensajes del handshake TLS
<code>tls.handshake.type == 1</code>	Solo ClientHello
<code>tls.handshake.type == 2</code>	Solo ServerHello
<code>tls.record.content_type == 23</code>	Solo Application Data (datos encriptados)
<code>tcp.port == 9001</code>	Todo el tráfico del puerto del taller

4. Parte 2 — DTLS sobre UDP

4.1 Sobre DTLS

DTLS (Datagram TLS) resuelve el problema de encriptar tráfico UDP. No podemos usar TLS directamente sobre UDP porque TLS asume un transporte ordenado y confiable. DTLS adapta TLS para datagramas:

- Añade números de secuencia explícitos a cada registro.
- Incluye su propio mecanismo de retransmisión para el handshake (ya que UDP no retransmite).
- Cada datagrama se encripta de forma independiente — si se pierde uno, los demás siguen siendo descifrables.

4.2 Qué esperar ver en Wireshark

A diferencia de TCP+TLS, aquí **no hay handshake TCP** (no hay SYN/SYN-ACK/ACK). El handshake DTLS ocurre directamente sobre UDP:

```

CLIENTE                                SERVIDOR
|                                     |
|- ClientHello (UDP) -->| handshake DTLS
|< HelloVerifyRequest --| (cookie anti-DoS)
|- ClientHello + cookie -->|
|< ServerHello, Certificate --|
|< ServerKeyExchange, Done |
|- ClientKeyExchange -->|
|- ChangeCipherSpec, Finished -->|
|< ChangeCipherSpec, Finished |
|                                     |
|- Application Data (UDP) >| datos encriptados
|< Application Data (UDP) -|
|                                     |

```

Note el paso adicional de `HelloVerifyRequest` con cookie — esto existe en DTLS pero no en TLS, y es un mecanismo para prevenir ataques de amplificación por reflexión (un problema específico de UDP, ya que no hay handshake de conexión previo).

4.3 Enfoque de implementación

Nota técnica: Las librerías de DTLS en Python (`pyOpenSSL`, `python-dtls`) tienen implementaciones experimentales que presentan errores frecuentes con versiones modernas de Python y OpenSSL. Para garantizar que el taller funcione sin problemas, usaremos un enfoque híbrido: las herramientas de línea de comandos de OpenSSL para el canal DTLS (que son muy estables y robustas) envueltas en scripts Python que automatizan la interacción. Esto tiene un valor pedagógico adicional: los estudiantes ven directamente las herramientas que implementan el protocolo, no una abstracción que oculta los detalles.

4.4 Servidor DTLS

Cree el archivo `dtls_servidor.py`:

```
import subprocess
import sys
import signal
import os

HOST = '127.0.0.1'
PORT = 9002

def main():
    print(f'Iniciando servidor DTLS en {HOST}:{PORT}')
    print('Esperando conexiones... (Ctrl+C para terminar)')
    print()

    while True:
        try:
            # openssl s_server con DTLS:
            # -dtls          : usar DTLS en lugar de TLS
            # -accept        : puerto donde escuchar
            # -cert/-key     : certificado y clave privada
            # -msg           : mostrar los mensajes del protocolo (útil
para depurar)
            proceso = subprocess.Popen(
                [
                    'openssl', 's_server',
                    '-dtls',
                    '-accept', str(PORT),
                    '-cert', 'certificado.pem',
                    '-key', 'clave.pem',
                    '-msg'
                ],
                stdin=subprocess.PIPE,
                stdout=subprocess.PIPE,
                stderr=subprocess.PIPE,
                text=True,
                bufsize=1
            )

            print(f'Servidor DTLS listo en el puerto {PORT}')

            # Leer la salida del servidor línea por línea
```

```

while True:
    linea = proceso.stdout.readline()
    if not linea and proceso.poll() is not None:
        break
    if linea.strip():
        # Filtrar líneas relevantes para el estudiante
        if any(clave in linea for clave in [
            'read from', 'write to', 'DONE',
            'Secure Renegotiation', 'Protocol',
            'Cipher', 'Session-ID'
        ]):
            print(f' {linea.strip()}')
        elif not linea.startswith(' '):
            # Datos recibidos del cliente
            mensaje = linea.strip()
            if mensaje:
                print(f'Recibido: {mensaje}')
                # Enviar respuesta
                respuesta = f'Servidor recibio: {mensaje}\n'
                try:
                    proceso.stdin.write(respuesta)
                    proceso.stdin.flush()
                except BrokenPipeError:
                    break

except KeyboardInterrupt:
    print('\nServidor detenido.')
    break
finally:
    try:
        proceso.terminate()
        proceso.wait(timeout=2)
    except:
        proceso.kill()

if __name__ == '__main__':
    main()

```

4.5 Cliente DTLS

Cree el archivo `dtls_cliente.py`:

```

import subprocess
import time
import sys
import threading

SERVIDOR = '127.0.0.1'
PORT = 9002

def leer_respuestas(proceso):

```

```

"""Hilo que lee las respuestas del servidor y las imprime."""
try:
    while True:
        linea = proceso.stdout.readline()
        if not linea and proceso.poll() is not None:
            break
        texto = linea.strip()
        if texto and not
texto.startswith(('>>>', '<<<', 'depth', 'verify')):
        # Ignorar líneas de depuración del protocolo, mostrar datos
        if 'Servidor recibio' in texto:
            print(f'Respuesta: {texto}')

except:
    pass

def main():
    print(f'Conectando al servidor DTLS en {SERVIDOR}:{PORT}...')

    # openssl s_client con DTLS:
    #   -dtls          : usar DTLS en lugar de TLS
    #   -connect       : dirección:puerto del servidor
    #   -CAfile        : certificado CA para verificar el servidor
    #   -msg           : mostrar mensajes del protocolo
    proceso = subprocess.Popen(
        [
            'openssl', 's_client',
            '-dtls',
            '-connect', f'{SERVIDOR}:{PORT}',
            '-CAfile', 'certificado.pem',
            '-msg'
        ],
        stdin=subprocess.PIPE,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True,
        bufsize=1
    )

    # Dar tiempo al handshake DTLS
    time.sleep(1)

    if proceso.poll() is not None:
        # El proceso terminó prematuramente
        error = proceso.stderr.read()
        print(f'Error al conectar: {error}')
        return

    print('Handshake DTLS completado')
    print()

    # Hilo para leer respuestas del servidor
    hilo_lector = threading.Thread(target=leer_respuestas, args=(proceso,),
    daemon=True)
    hilo_lector.start()

```



```

mensajes = [
    'Hola, este es el primer mensaje seguro por UDP',
    'Este datagrama viaja encriptado',
    'Cada paquete se encripta de forma independiente',
    'Ultimo mensaje, adios'
]

try:
    for msg in mensajes:
        print(f'Enviado: {msg}')
        proceso.stdin.write(msg + '\n')
        proceso.stdin.flush()
        time.sleep(1) # separar mensajes en la traza

except BrokenPipeError:
    print('Conexión cerrada por el servidor.')
finally:
    try:
        proceso.stdin.close()
        proceso.wait(timeout=3)
    except:
        proceso.kill()

print()
print('Sesión DTLS cerrada')

if __name__ == '__main__':
    main()

```

4.6 Alternativa: ejecución manual con OpenSSL

Si prefiere ejecutar los comandos directamente sin los scripts Python, puede hacerlo desde dos terminales:

Terminal 1 (servidor):

```
openssl s_server -dtls -accept 9002 -cert certificado.pem -key clave.pem
```

Terminal 2 (cliente):

```
openssl s_client -dtls -connect 127.0.0.1:9002 -CAfile certificado.pem
```

Una vez conectados, escriba un mensaje en la terminal del cliente y presione Enter. Verá que el servidor recibe el mensaje y puede responder escribiendo en su propia terminal. Para cerrar la conexión, escriba **Q** y presione Enter, o presione **Ctrl+C**.

¿Por qué usar OpenSSL directamente? Las herramientas **s_server** y **s_client** son parte de la implementación de referencia de TLS/DTLS. Cuando usted ejecuta **openssl s_server -dtls**, está

usando exactamente el mismo código criptográfico que usan los servidores web, las VPN, y las aplicaciones de producción. Es la forma más directa de ver el protocolo en acción.

4.7 Instrucciones de ejecución y captura

1. En Wireshark, inicie una nueva captura en la interfaz correspondiente.
2. Aplique el filtro de captura: `udp port 9002`.
3. En una terminal: `python3 dtls_servidor.py` (o el comando `openssl s_server` manual).
4. En otra terminal: `python3 dtls_cliente.py` (o el comando `openssl s_client` manual).
5. Cuando termine, detenga la captura y guárdela como `captura_dtls.pcap`.

Filtros útiles para Wireshark:

Filtro	Qué muestra
<code>dtls</code>	Solo paquetes DTLS
<code>dtls.handshake</code>	Solo mensajes del handshake DTLS
<code>dtls.handshake.type == 1</code>	Solo ClientHello
<code>dtls.handshake.type == 3</code>	Solo HelloVerifyRequest (exclusivo de DTLS)
<code>dtls.record.content_type == 23</code>	Solo Application Data
<code>udp.port == 9002</code>	Todo el tráfico del puerto

5. Parte 3 — Comparación

5.1 Actividad: comparar las cuatro capturas

Si conservan las capturas del taller anterior (`captura_tcp.pcap` y `captura_udp.pcap`), pueden abrir las cuatro capturas en Wireshark y compararlas directamente.

Ejercicio guiado: Para cada protocolo (TCP plano, UDP plano, TLS, DTLS), busque en la traza y anote:

| Aspecto | TCP plano | UDP plano | TLS (TCP) | DTLS (UDP) | ||--|| | ¿Cuántos paquetes antes del primer dato? | |||| | ¿El contenido del mensaje es legible? | |||| | ¿Qué protocolo muestra Wireshark en la columna Protocol? | |||| | ¿Hay un handshake visible? ¿Cuántos paquetes tiene? | |||| | ¿Se ve un mensaje de tipo "HelloVerifyRequest"? | |||| | ¿Hay paquetes de cierre de conexión? ¿Cómo se ven? | |||| |

5.2 Actividad: intentar leer los datos

1. En la captura TCP del taller anterior, haga clic derecho sobre un paquete de datos → **Follow** → **TCP Stream**. Los mensajes de texto son completamente legibles.
2. Repita lo mismo en la captura TLS: haga clic derecho sobre un paquete de "Application Data" → **Follow** → **TCP Stream**. ¿Qué ve ahora?
3. En la captura UDP del taller anterior, seleccione un paquete de datos y observe el panel inferior de Wireshark. El contenido es visible en la sección "Data".
4. Repita con la captura DTLS. ¿Puede leer el contenido?

6. Cuestionario

Responda las siguientes preguntas basándose en las trazas capturadas.

Sobre TLS:

1. En la captura TLS, ¿cuántos paquetes hay entre el último ACK del handshake TCP y el primer paquete de "Application Data"? Estos paquetes corresponden al handshake TLS. Liste los tipos de mensaje que observa.
2. Abra el paquete ClientHello y expanda los detalles en Wireshark. ¿Qué información envía el cliente en este mensaje? ¿Está encriptada esta información?
3. Abra el paquete ServerHello. ¿Qué versión de TLS y qué cipher suite se negociaron? Anótelos.
4. Después del handshake, seleccione un paquete de "Application Data". ¿Puede ver el mensaje original ("Hola, este es el primer mensaje seguro")? ¿Por qué sí o por qué no?
5. Aunque el contenido está encriptado, ¿qué información sobre la comunicación sigue siendo visible en la traza? (Piense en direcciones, puertos, tamaños, tiempos.)

Sobre DTLS:

6. En la captura DTLS, ¿hay paquetes SYN o ACK antes del handshake? ¿Por qué?
7. ¿Aparece un mensaje HelloVerifyRequest en la traza? ¿Cuál es el propósito de este mensaje? ¿Por qué TLS no lo necesita pero DTLS sí?
8. Compare el número total de paquetes del handshake DTLS con el handshake TLS. ¿Cuál tiene más? ¿A qué se debe la diferencia?
9. En los paquetes DTLS, busque los campos `epoch` y `sequence_number` en los detalles del registro DTLS. ¿Existen campos equivalentes en TLS? ¿Por qué DTLS los necesita?

Comparación general:

10. Complete la siguiente tabla midiendo los tiempos en sus trazas:

Métrica	TCP plano	TLS	UDP plano	DTLS	-- --	Tiempo hasta el primer dato enviado (ms)	
Número total de paquetes en la sesión					Tamaño promedio de los paquetes de datos (bytes)		

11. TLS agrega overhead tanto en tiempo (latencia adicional) como en espacio (bytes adicionales por paquete). Basándose en sus mediciones, ¿cuántos bytes adicionales agrega TLS por paquete comparado con TCP plano?
12. Proponga dos escenarios reales donde usaría DTLS en lugar de TLS. Justifique considerando las características que observó en las trazas.

7. Entregables

- `certificado.pem` y `clave.pem` generados.
- `tls_servidor.py` y `tls_cliente.py` funcionando.

- `dtls_servidor.py` y `dtls_cliente.py` funcionando (o evidencia de ejecución con `openssl s_server / s_client`).
- `captura_tls.pcap` con la sesión TLS completa.
- `captura_dtls.pcap` con la sesión DTLS completa.
- Este documento con el cuestionario respondido.
- Capturas de pantalla de los paquetes ClientHello (TLS y DTLS) expandidos en Wireshark.

8. Rúbrica

| Componente | Peso | || | Código TLS funcional (servidor + cliente) | 20% | | Código DTLS funcional (servidor + cliente) | 20% | | Capturas .pcap con tráfico completo | 20% | | Cuestionario respondido | 30% | | Tabla comparativa completa (pregunta 10) | 10% |

9. Recursos adicionales

- RFC 8446 — TLS 1.3: <https://www.rfc-editor.org/rfc/rfc8446>
- RFC 9147 — DTLS 1.3: <https://www.rfc-editor.org/rfc/rfc9147>
- Documentación del módulo `ssl` de Python: <https://docs.python.org/3/library/ssl.html>
- Documentación de `openssl s_server`: https://www.openssl.org/docs/man3.0/man1/openssl-s_server.html
- Documentación de `openssl s_client`: https://www.openssl.org/docs/man3.0/man1/openssl-s_client.html
- Wiki de Wireshark sobre TLS: <https://wiki.wireshark.org/TLS>
- Wiki de Wireshark sobre DTLS: <https://wiki.wireshark.org/DTLS>